

HYTech LMS Developer Turnover Guide

HYTech LMS Developer Turnover Guide

System: HYTech Learning Management System

Organization: HYT Global Institute

Document type: Technical handover, setup, operations, security, and maintenance guide

Repository application root: `HYTech/`

Audited against repository: July 29, 2026

Classification: Internal technical documentation

This guide was rebuilt from the application source, Firebase rules and configuration, Functions, tests, scripts, and CI/CD workflows. It describes the checked-out repository on the audit date. Live Firebase state can differ; verify the target project and current risk register before an operational change.

1. Purpose and handover position

HYTech LMS is a React single-page learning management system for three active roles:

- **Administrator:** manages users, sectors, course templates, classes, settings, logs, ID requests, incidents, and notifications.
- **Trainer:** delivers classes, manages trainees and co-trainers, publishes learning content, creates assessments and submission tasks, grades work, and graduates trainees.
- **Student/trainee:** registers, joins classes, consumes content, completes assessments and tasks, receives notifications, and tracks progress.

The system uses Firebase Authentication, Cloud Firestore, Cloud Storage, Cloud Functions, App Check, and Firebase Hosting. Most normal reads and selected writes go directly from the browser to Firebase under Security Rules. Operations that must be authoritative, privileged, recursive, or race-safe use callable Cloud Functions.

This is not a finished risk-free handover. Read these companion documents before accepting production responsibility:

- `HYTech/docs/TURNOVER_RISK_REGISTER.md`
- `HYTech/docs/SECURITY_REMEDIATION_OPERATIONS.md`
- `HYTech/docs/DEPENDENCY_RISK_REGISTER.md`
- `docs/FIREBASE_ENVIRONMENTS.md`
- `docs/FULL_WEBSITE_QA_PLAYBOOK.md`

The risk register is a dated operational record. Do not copy its claims into a new release without checking the live projects again.

2. System at a glance

```
User browser
|
+-- Firebase Hosting: Vite-generated React SPA
|
+-- Firebase Authentication: email/password identity
+-- Cloud Firestore: profiles, classes, content, enrollment, activity
+-- Cloud Storage: avatars, branding, class files, submissions
+-- App Check: reCAPTCHA v3 attestation
+-- Callable Functions: trusted grading, progress, graduation, admin work
`-- Firestore triggers: directory, notifications, enrollment counts
```

Environments

Environment	Firebase project	Intended use
Production	hyt-global-institute-lms	Live service and real records
Staging	hytech-lms-staging	Integration, role QA, and release validation
Emulator/demo	demo-hytech-lms	Local Firestore and Storage rule tests

The `.firebaserc` default is production. Always supply `--project staging` or use the scoped npm scripts during development.

Technology baseline

Area	Repository baseline
UI	React 18, React Router 7, Tailwind CSS 3, Lucide React
Build	Vite 8
Browser SDK	Firebase Web SDK 12
Backend	Firebase Functions v1 API on Node.js 22
Backend dependencies	firebase-functions 7, firebase-admin 13
Test tools	Vitest, Firebase Rules Unit Testing, Playwright, axe-core
Delivery	GitHub Actions and Firebase CLI 15.24.0
Primary region	asia-southeast1

Use the lockfiles as the source of exact resolved dependency versions. The semver ranges in `package.json` are not a reproducible bill of materials.

3. Repository map

```
HYTechLMS/
|-- .github/workflows/
|   |-- qa.yml # CI security, unit, rules, build, E2E
|   `-- deploy.yml # Gated full production deployment
```

```

|-- docs/                                # Environment, QA, design, roadmap audits
|-- HYTech/                              # Run application commands here
|   |-- docs/                            # Security and turnover risk runbooks
|   |-- functions/
|       |-- src/index.js                 # Deployed Functions (single module)
|       |-- package.json
|       |-- package-lock.json
|   |-- public/                          # Files copied directly into dist
|   |-- scripts/                         # QA, backup, secret, migration utilities
|   |-- src/
|       |-- components/                  # Screens grouped by role/domain
|       |-- context/                     # Auth, toast, settings, notifications
|       |-- data/tesdaCatalog.js         # Local reference catalog
|       |-- hooks/
|       |-- utils/firestoreService.js    # Main browser data-access module
|       |-- App.jsx                      # Route table
|       |-- firebase.js                  # Firebase/App Check initialization
|       |-- main.jsx                     # React entry point
|   |-- tests/
|       |-- e2e/
|       |-- rules/
|       |-- unit/
|   |-- firebase.json                    # Hosting, Firestore, Storage, Functions
|   |-- firestore.rules
|   |-- firestore.indexes.json
|   |-- storage.rules
|   |-- playwright.config.js
|   |-- vite.config.js
|   |-- package.json
|-- HYTECH_LMS_DEVELOPER_TURNOVER_GUIDE.md
`-- HYTECH_LMS_DEVELOPER_TURNOVER_GUIDE.pdf

```

Read these source files first

1. HYTech/src/App.jsx
2. HYTech/src/context/AuthContext.jsx
3. HYTech/src/components/auth/RoleProtectedRoute.jsx
4. HYTech/src/firebase.js
5. HYTech/src/utils/firestoreService.js
6. HYTech/functions/src/index.js
7. HYTech/firestore.rules
8. HYTech/storage.rules
9. .github/workflows/qa.yml
10. .github/workflows/deploy.yml

Maintainability hotspots

The largest source modules are:

- `ClassDetail.jsx` : approximately 7,700 lines
- `firestoreService.js` : approximately 5,100 lines
- `StudentCourse.jsx` : approximately 4,000 lines
- `functions/src/index.js` : approximately 1,950 lines

These are change-risk concentrations. Split by domain with regression tests before adding another major feature. Avoid opportunistic rewrites during a security or production incident.

4. Product behavior by role

Administrator

- View system totals and dashboard activity.
- Create and update Auth-backed user accounts.
- Assign the active `admin`, `trainer`, and `student` roles.
- Activate or disable application profiles.
- Manage TESDA-oriented sectors and reusable course templates.
- Create classes and assign lead/co-trainers.
- Preview a class through the student course UI.
- Review and purge application activity logs.
- Review ID requests and incident forms.
- Configure branding, public access, and notification settings.
- Read role/user-targeted notifications.

Trainer

- View active and archived classes.
- Create a class and work from reusable template content.
- Manage lead ownership and co-trainers.
- Review pending join requests and the class roster.
- Approve, change, remove, or graduate enrollments.
- Organize topics, modules, materials, and announcements.
- Upload files and manage announcement comments.
- Create quiz/form assessments and manual submission assignments.
- Configure points, attempts, dates, duration, passing scores, and publishing.
- Review quiz attempts, manually grade pending responses, and grade submissions.
- Inspect class gradebook and class activity.
- Manage class presentation details and meeting links.
- Review incident forms.

Student/trainee

- Self-register as a student and verify an email address.
- Complete a forced password change when an admin-created profile requires it.
- Browse joinable classes or request entry using a class code.
- Wait for trainer/admin approval before becoming a class member.
- View current and archived classes.

- Personalize class-card colors, images, and nicknames.
- View announcements, topics, modules, materials, and activity.
- Comment on announcements.
- Complete securely graded assessments.
- Submit text, links, and permitted files for manual tasks.
- View tasks, deadlines, a personal calendar, notifications, and progress.
- Submit an ID request or own incident form.

Implemented but not currently exposed

- Server-owned certificate issuance, revocation, and public token verification are implemented.
- Public verification is routed at `/verify/:token`.
- `StudentCertificates.jsx` exists, but `/student/certificates` currently redirects to `/student`, and its sidebar entry is commented out.
- Several legacy lifecycle handlers remain as non-exported constants at the end of `functions/src/index.js`; they are not deployed entry points. Do not assume their comments describe active behavior.
- No `supervisor` route or active product role exists, even if old documents or historical data mention one.

5. Frontend architecture

Entry, routing, and code splitting

`src/main.jsx` renders `App`. `App.jsx` lazy-loads route screens under a single `Suspense` boundary. Firebase Hosting rewrites all unknown asset-independent paths to `index.html`, so React Router owns application routing.

Route	Access/purpose
<code>/</code>	Public landing page
<code>/signup</code>	Public-only registration
<code>/signin</code>	Public-only sign-in
<code>/verify-email</code>	Email verification
<code>/verify/:token</code>	Public certificate verification
<code>/admin</code>	Admin dashboard
<code>/admin/users</code>	User management
<code>/admin/sectors</code>	Sectors/templates
<code>/admin/classes</code>	Class administration
<code>/admin/classes/:classname/preview</code>	Student-view preview
<code>/admin/logs</code>	Application activity logs
<code>/admin/id-requests</code>	ID requests
<code>/admin/incident-forms</code>	All incidents
<code>/admin/settings</code>	Global settings
<code>/admin/notifications</code>	Admin notifications

Route	Access/purpose
/trainer	Trainer home
/trainer/:className	Class delivery and management
/trainer/tasks	Trainer task view
/trainer/sectors/:sectorId	Sector detail
/trainer/archived	Archived classes
/trainer/settings	Trainer settings
/trainer/notifications	Trainer notifications
/trainer/incident-forms	Incident review
/student	Student home/waiting-room flow
/student/enroll	Enrollment browser
/student/calendar	Deadline and personal calendar
/student/request-id	ID request
/student/incident-form	Own incidents
/student/:classname	Class learning view
/student/tasks	Cross-class tasks
/student/archived	Completed classes
/student/settings	Student settings
/student/notifications	Student notifications
/class/:className	Authenticated shared/legacy class entry

Unknown paths return to /. Route order matters because :className is a dynamic child under trainer and student layouts.

Authentication and authorization flow

Identity and authorization are separate:

1. Firebase Authentication establishes the UID.
2. users/{uid} supplies application role, status, and profile state.
3. AuthContext tracks the Auth user and matching profile.
4. public/authenticated/role guards control navigation.
5. email verification and forced-password-change state add gates.
6. Firestore and Storage Rules authorize direct Firebase operations.
7. callable Functions repeat identity, active-status, role, ownership, and input checks for privileged operations.

Never treat a hidden control or route guard as a security boundary. Browser code and all VITE_* values are public.

Self-registration must not allow role selection. Administrative user creation must use the supported account workflow so Firebase Auth and Firestore stay consistent.

State and data access

There is no Redux-style store. The application uses:

- React component state for screen-level behavior.
- `AuthContext` for identity/profile.
- `ToastContext` for user feedback.
- `useAppSettings`, `useProfileAvatar`, `useRoleNotifications`, and `useUserSettings`.
- `useFirestoreQuery` and direct `onSnapshot` subscriptions.
- `firestoreService.js` as the main browser-side service layer.

Preferred change pattern:

```
Screen/component
-> named service function
-> Firestore/Storage/Callable API
-> Rules or Function authorize
-> server timestamp / transaction where required
-> refresh or snapshot update
```

Do not add new direct Firestore calls to large components if an operation belongs in the service layer or trusted backend.

Firestore client initialization

`src/firebase.js` reads:

```
VITE_FIREBASE_API_KEY=
VITE_FIREBASE_AUTH_DOMAIN=
VITE_FIREBASE_PROJECT_ID=
VITE_FIREBASE_STORAGE_BUCKET=
VITE_FIREBASE_MESSAGING_SENDER_ID=
VITE_FIREBASE_APP_ID=
VITE_RECAPTCHA_SITE_KEY=
```

It initializes Auth, Firestore, Storage, Functions in `asia-southeast1`, and reCAPTCHA v3 App Check when configured. A Firebase web config and reCAPTCHA site key are identifiers embedded in the browser bundle, not server secrets.

This guide intentionally contains no API key or environment-specific web configuration value. Firebase describes its web API keys as project identifiers, not authorization credentials; Security Rules, IAM, and App Check protect data. Even so, the receiving team must obtain its own values from the authorized Firebase project, restrict the key to required Firebase APIs, and keep environment configuration out of turnover documentation.

6. Firestore data model

Firestore has no repository-enforced TypeScript schema. Treat this as a path and ownership map, not a guarantee that every historical document has every field. Inspect data before migration and preserve legacy compatibility deliberately.

Top-level collections

Path	Responsibility and trust
<code>users/{uid}</code>	Public application profile, role, status, identity display fields
<code>users/{uid}/private/{doc}</code>	Private profile information
<code>users/{uid}/lmsExperience/profile</code>	LMS preferences/experience counters
<code>users/{uid}/calendarEvents/{id}</code>	Student-owned personal events
<code>users/{uid}/classPrefs/{classId}</code>	Student-owned class-card preferences
<code>userSettings/{uid}</code>	User-specific UI/settings document
<code>config/appSettings</code>	Public application branding/access configuration
<code>sectors/{sectorId}</code>	Sector metadata
<code>courses/{courseId}</code>	Reusable course template and legacy course data
<code>classes/{classId}</code>	Live delivery instance
<code>classDirectory/{classId}</code>	Server-maintained class discovery projection
<code>enrollments/{id}</code>	Student/class relationship and lifecycle
<code>idRequests/{id}</code>	ID processing workflow
<code>incidentForms/{id}</code>	Incident submission and review
<code>notifications/{id}</code>	User/role notification
<code>activityLogs/{id}</code>	Application audit/activity record
<code>securityLogs/{id}</code>	Server-only security-sensitive event record
<code>students/{uid}/progress/{classId}</code>	Server-owned calculated progress
<code>certificates/{certificateId}</code>	Server-owned certificate record

`courseApplications` and legacy nested course content are referenced by old backend code/data paths. Verify whether any live documents remain before removing compatibility code.

Live class tree

```

classes/{classId}
|-- members/{uid}
|-- activity/{eventId}
|-- announcements/{announcementId}
|   |-- comments/{commentId}
|-- topics/{topicId}
|-- modules/{moduleId}
|   |-- materials/{materialId}
|-- materials/{materialId}
|-- assessments/{assessmentId}
|   |-- private/answerKey
|   |-- attempts/{attemptId}

```

```
`-- assignments/{assignmentId}
  |-- private/answerKey
  |-- attempts/{attemptId}
  `-- submissions/{studentId}
```

Template versus class

- A `course` is reusable template/legacy curriculum data.
- A `class` is a live delivery instance with trainers and enrollments.
- `promoteClassToTemplate` copies supported class content into a template.
- `cloneTemplateToClass` copies template content and referenced Storage objects into a class.
- Files are copied to new Storage paths; a template/class copy is not merely a shared document reference.

Assessments versus assignments

There are two quiz-capable authoring paths:

- `classes/{classId}/assessments/{id}`
- `classes/{classId}/assignments/{id}`

Assignments can also be submission tasks. Code that handles quiz attempts must carry the correct parent kind; attempts are stored below the item that authored them. Never infer the collection only from an item ID.

Correct answers belong only in `private/answerKey`. Public assessment documents must not contain inline answers. Browser clients cannot create or update attempt documents; `submitAssessmentAttempt` owns secure quiz submission.

Enrollment and academic state

The application uses historical status spelling/casing in some UI paths. Canonical comparisons commonly normalize values to lowercase. Active learning access generally recognizes `active` or `ongoing`; pending join requests are not members; completed enrollments appear as archived.

Trusted academic lifecycle:

```
Student work
-> server-written assessment attempts / controlled submissions
-> recalculateMyProgress
-> server-owned progress record
-> trainer/admin graduation decision
-> graduateEnrollment
-> completed enrollment + certificate
-> optional revokeCertificate
```

Do not reintroduce client-computed completion as authoritative evidence.

7. Cloud Functions

All deployed exports are in `functions/src/index.js`, use the Functions v1 API, and run in `asia-southeast1`.

Runtime groups and limits

Group	Configuration	Use
Ordinary	<code>maxInstances: 5</code> , App Check enforced	Most triggers/callables
Submission	<code>maxInstances: 40</code> , App Check enforced	Concurrent quiz submissions
Template copy	<code>maxInstances: 2</code> , 540 s, 1 GB, App Check enforced	Deep content/file copies

Do not add `minInstances` casually. The source documents a gen-1 regional pricing/deployment constraint and intentional avoidance of always-on cost.

Firestore triggers

Export	Trigger purpose
<code>syncClassDirectory</code>	Maintain sanitized class discovery data
<code>notifyTraineesOnAnnouncement</code>	Notify active trainees of announcements
<code>notifyTraineesOnAssessmentPublish</code>	Notify on assessment publication
<code>notifyTraineesOnAssignmentPublish</code>	Notify on assignment publication
<code>syncClassEnrollmentCount</code>	Maintain enrollment count on classes

Callable Functions

Export	Authorized responsibility
<code>migrateClassDirectory</code>	Admin repair/backfill of directory records
<code>adminUpdateUserAccount</code>	Admin update of Auth/profile account data
<code>submitAssessmentAttempt</code>	Student validation, attempt enforcement, secure grading
<code>recalculateMyProgress</code>	Rebuild caller progress from trusted records
<code>changeEnrollmentStatus</code>	Controlled staff enrollment transition
<code>graduateEnrollment</code>	Trusted completion and certificate issuance
<code>revokeCertificate</code>	Admin certificate revocation
<code>verifyCertificate</code>	Public token-based certificate verification
<code>gradeAssessmentAttempt</code>	Staff manual/final attempt grading
<code>migrateAssessmentAnswerKeys</code>	Admin migration of legacy inline answers
<code>promoteClassToTemplate</code>	Copy live class content into a template
<code>cloneTemplateToClass</code>	Copy template content/files into a live class
<code>deleteAssessmentSecure</code>	Authorized recursive assessment deletion
<code>deleteClassSecure</code>	Authorized recursive class deletion
<code>deleteCourseTemplateSecure</code>	Authorized recursive template deletion
<code>deleteSectorSecure</code>	Delete an unused sector after dependency checks

Function engineering rules

- Treat every callable payload as hostile.
- Require Auth unless the operation is intentionally public.
- Load the caller profile; verify active status, role, and object ownership.
- Check class lead/co-trainer relationships server-side.
- Bound strings, arrays, numeric ranges, attempts, file references, and dates.
- Use server timestamps, transactions, or batches for sensitive state changes.
- Keep answer keys, certificate records, and security logs server-owned.
- Write security logs for privileged academic/account changes.
- Never return private answer material beyond explicitly permitted result data.
- Test both assessment parent kinds.
- Load the Functions module in CI; syntax-only validation does not detect incompatible Admin SDK APIs.
- Confirm new callables use an App Check-enforced runtime wrapper.

The non-exported `legacyCleanupUserData` and `legacyCleanupCourseData` constants near the end of the file are not active deployed exports. Account deletion therefore does not currently guarantee cleanup of Firestore data.

8. Security model

Defense layers

UI/route guard

-> Firebase Authentication

-> active users/{uid} profile

-> Firestore or Storage Security Rules

-> App Check attestation

-> callable validation/transaction for privileged operations

-> activity/security logging

No single layer replaces the others.

Firestore rule posture

Key properties of the current rules:

- user profile role/status escalation is constrained;
- self-owned settings, calendar events, and preferences are scoped to the UID;
- class reads require admin, assigned trainer/co-trainer, or membership;
- class content writes are limited to class staff;
- private answer keys are limited to staff;
- quiz attempts are created by Functions, not clients;
- students can manage only their own eligible task submission;
- enrollment creation/transitions are field- and role-constrained;
- progress, certificates, and security logs are server-owned;
- `classDirectory` is server-written;
- top-level course/sector deletion is blocked from direct clients and routed to controlled Functions.

Queries must satisfy rule constraints. A rule is not a post-query filter: Firestore rejects a query that could return unauthorized documents.

Storage rule posture

Supported roots:

```
userAvatars/{uid}/{role}/{fileName}
lmsFiles/{classId}/{uid}/{...}
branding/{fileName}
```

- avatar writes require the same UID, claimed path role, allowed image type, and size limit;
- class files require active LMS access, class staff/membership as applicable, owner-scoped writes, permitted content types, and size limits;
- branding is publicly readable and admin-written;
- all unmatched paths are denied.

Storage rules validate declared metadata/MIME and size, not file contents. There is no malware scanning or quarantine pipeline.

Hosting security

`firebase.json` configures:

- HSTS;
- MIME sniffing prevention;
- frame denial;
- strict referrer policy;
- restrictive browser permissions;
- cross-origin opener policy;
- Content Security Policy;
- immutable caching for hashed assets;
- no-cache for `index.html`;
- SPA fallback to `index.html`.

When adding a third-party API, image host, iframe, analytics script, or font, update CSP narrowly and test production headers. Do not solve a CSP error with a wildcard.

Secrets and privacy

Never commit:

- service-account JSON;
- `FIREBASE_TOKEN`;
- production or staging passwords;
- `.env.local`, `.env.staging.local`, `.env.e2e.local`;
- App Check debug tokens;
- exported learner data or audit output.

`VITE_*` values are compiled into JavaScript and must never hold a secret. Backups and audit reports can contain personal or academic data; write them to an approved restricted location, not the repository.

Run:

```
npm run security:secrets
```

This scanner deliberately covers the repository outside `HYTech/` as well.

9. Local developer setup

Access required

Obtain:

- authorized repository access;
- staging Firebase access;
- production access only if responsible for releases/operations;
- dedicated staging accounts for admin, trainer, enrolled student, unenrolled student, pending student, and disabled student as needed;
- permission to manage environment variables/secrets only when required.

Do not exchange account passwords or service-account files in chat.

Prerequisites

- Git
- Node.js 22 LTS (matches Functions and CI)
- npm
- Java 21 for Firebase emulator tests
- Firebase CLI
- Playwright browser binaries

Check:

```
git --version
node --version
npm --version
java --version
firebase --version
```

Do not use an arbitrary newer local Node version as release evidence. Use Node 22 when reproducing CI or diagnosing Functions.

Install

From the repository:

```
Set-Location HYTech
npm ci
npm ci --prefix functions
npx playwright install chromium
```

Install all E2E engines only for full cross-browser work:

```
npx playwright install
```

Use `npm ci`, not `npm update`, for initial setup.

Configure staging

If no populated file exists:

```
Copy-Item .env.staging.example .env.staging.local
```

Then fill the staging Firebase web configuration and reCAPTCHA site key. If the file already exists, edit/add missing keys; copying the example will overwrite it with blanks.

Default `npm run dev` and `npm run build` use the default Vite environment, normally `.env.local`. Repository operations documentation warns that this may point to production. Prefer:

```
npm run dev:staging
```

```
npm run build:staging
```

Obtain the Firebase web configuration

Do not copy values from this guide, another developer's workstation, a screenshot, an old deployment, or another environment.

For each environment:

1. Sign in to the [Firebase console](#) using an organization-managed account with authorized project access.
2. Select the exact staging or production project.
3. Open **Project settings** (gear icon) and select **General**.
4. In **Your apps**, select the registered HYTech web app. If it does not exist, click **Add app**, choose Web, give it an environment-specific display name, and register it.
5. Under **SDK setup and configuration**, choose **Config**.
6. Map the displayed fields into the local ignored environment file:

```
VITE_FIREBASE_API_KEY=<apiKey shown for this web app>
VITE_FIREBASE_AUTH_DOMAIN=<authDomain shown for this web app>
VITE_FIREBASE_PROJECT_ID=<projectId shown for this web app>
VITE_FIREBASE_STORAGE_BUCKET=<storageBucket shown for this web app>
VITE_FIREBASE_MESSAGING_SENDER_ID=<messagingSenderId shown for this web app>
VITE_FIREBASE_APP_ID=<appId shown for this web app>
VITE_RECAPTCHA_SITE_KEY=<site key from this environment's App Check registration>
```

7. Verify `VITE_FIREBASE_PROJECT_ID` before starting Vite or building.
8. In Google Cloud Console, open **APIs & Services > Credentials**, select the Firebase-created browser key, and confirm its API restrictions permit only required Firebase APIs. Use a different restricted key for any future non-Firebase API. Never add a paid generative-AI, Maps, or unrelated API to the Firebase browser key's allowlist.
9. Put CI values in protected GitHub environment variables/secrets through repository settings. Do not paste them into workflow YAML or documentation.

The web configuration does not grant database access by itself. A successful connection with unsafe rules is still unsafe; hiding the key does not repair unsafe rules.

Configure E2E

Copy `.env.e2e.example` to `.env.e2e.local` and supply dedicated staging-only accounts:

```
E2E_BASE_URL=https://hytech-lms-staging.web.app
E2E_ADMIN_EMAIL=
E2E_ADMIN_PASSWORD=
E2E_TRAINER_EMAIL=
E2E_TRAINER_PASSWORD=
E2E_STUDENT_EMAIL=
E2E_STUDENT_PASSWORD=
```

The CI suite additionally supports unenrolled, pending, and disabled student credentials. `E2E_REQUIRE_AUTH=true` makes missing required accounts fail rather than skip.

`playwright.config.js` rejects unexpected remote targets. Local preview is accepted only with `E2E_ALLOW_LOCAL=true`.

First local verification

```
npm run security:secrets
npm run test:unit
npm run test:rules
node --check functions/src/index.js
node -e "require('./functions/src/index.js')"
npm run build:staging
npm run test:e2e:public
```

Rules tests start Firestore and Storage emulators through Firebase CLI and use the `demo-hytech-lms` project. Java must be installed.

10. Creating a new Firebase environment

Use this section when rebuilding staging, creating replacement infrastructure, or performing a recovery exercise. Console labels, quotas, and pricing can change; check the linked official documentation before approval.

10.1 Decide ownership and environment separation

Recommended structure:

- a separate Firebase/Google Cloud project for production and staging;
- optional short-lived projects for recovery testing;
- organization-managed accounts and IAM groups, not personal ownership;
- separate Auth users, web apps, App Check registrations, databases, buckets, budgets, service identities, domains, and CI environments;
- least-privileged access with at least two recoverable owners;
- no production learner data in staging.

Do not combine staging and production in one Firebase project. Billing, Auth, Firestore, Storage, Functions, quota, and many controls are project-scoped. Separation reduces accidental access and improves incident containment.

Before provisioning, record:

- legal, technical, billing, and incident-response owners;
- approved budget and expected trainee/file/assessment volume;
- data residency and region decision;
- retention, backup, RPO, and RTO requirements;
- production domain and DNS owner;
- privacy and academic approval authority.

10.2 Create the projects

For staging and then production:

1. In Firebase Console, select **Add project**.
2. Use an approved project name and permanent project ID.
3. Attach it to the correct Google Cloud organization/folder where available.
4. Enable Google Analytics only if there is an approved requirement and privacy basis; core HYTech LMS operation does not depend on it.
5. Grant required IAM roles to organization groups.
6. Remove broad temporary setup access.
7. Add Firebase CLI aliases from `HYTech/` :

```
firebase login
firebase projects:list
firebase use --add <staging-project-id>
firebase use --add <production-project-id>
```

Name the aliases `staging` and `production`.

Project creation is quota-limited, and complete deletion takes time. Give every temporary project a cleanup owner and date.

10.3 Select billing and establish cost controls

Firebase provides the no-cost **Spark** plan and pay-as-you-go **Blaze** plan. HYTech LMS uses Cloud Storage and Cloud Functions. Current Firebase documentation requires Blaze for Cloud Storage and provides Functions access on Blaze, so plan staging and production as billable projects.

Linking a Cloud Billing account upgrades the entire project. Before deployment:

1. have the billing owner link the approved billing account;
2. open **Google Cloud Console > Billing > Budgets & alerts**;
3. create a monthly budget scoped to each project;
4. route alerts to engineering and billing owners;
5. use several thresholds, such as 25%, 50%, 75%, 90%, 100%, plus a forecasted 100% alert;
6. retain the checked-in Function `maxInstances` limits;
7. review cost and quota dashboards after staging load tests and monthly after launch.

Ordinary budgets alert; they do not cap usage or guarantee that charges stop. Automated billing shutdown can interrupt sign-in, grading, uploads, and trusted academic writes. Use it only with an approved failure and recovery plan.

Do not promise a fixed monthly price in this guide. Cost depends on region, reads and index scans, writes, stored/downloaded files, Function runtime and memory, Hosting traffic, logs, backups/PITR, taxes, and currency. Estimate with the official pricing calculator and measured staging traffic.

10.4 Cost drivers and recommendations

Service	Main cost drivers	Recommendation
Authentication	active users and provider/tier	retain email/password unless another provider is approved; monitor active-user limits
Firestore	document/index reads, writes, deletes, storage, bandwidth	paginate; avoid collection scans and unbounded live listeners
Storage	stored bytes, operations, downloads, retained versions	enforce limits; compress images; apply an approved lifecycle policy
Functions	invocations, compute time, memory, networking, artifacts	retain instance ceilings; bound retries and fan-out
Hosting	stored data and outbound transfer	retain immutable asset caching and control bundle size
App Check	provider assessments beyond its allowance	monitor usage; do not shorten token TTL without need
Backup/PITR	retained data, restores, clones	budget explicitly; test restore; expire under approved retention
Logging	ingestion and retained log volume	never log sensitive payloads; configure deliberate retention

At the audit date, official Firestore documentation lists one free-quota database per project with 1 GiB stored data, 50,000 reads/day, 20,000 writes/day, 20,000 deletes/day, and 10 GiB outbound transfer/month. TTL deletes, PITR, backups, restores, and clones are billable and do not receive this free usage. Recheck current quotas before budgeting.

Cost design recommendations:

- unsubscribe Firestore listeners on component unmount;
- paginate activity, notifications, logs, submissions, and gradebook data;
- store files in Storage, not base64 payloads in Firestore;
- prevent retry loops and duplicate notification fan-out;
- clean disposable staging fixtures and obsolete build artifacts;
- use emulators instead of live projects for rule tests;
- alert on sudden read, egress, Function, and Storage growth;
- review cost per active trainee, not only the total invoice.

10.5 Register a web app and obtain its configuration

1. Open **Project settings > General > Your apps**.
2. Register one Web app for the environment.
3. Give it an unambiguous name such as **HYTech LMS - Staging**.
4. Retrieve its values using **Obtain the Firebase web configuration** above.
5. Register only legitimate development, staging, and production domains.

Registering a web app does not create Auth, Firestore, Storage, Functions, or Hosting. Provision each product explicitly.

10.6 Enable Authentication

1. Open **Build > Authentication** and click **Get started**.
2. Enable **Email/Password** under **Sign-in method**.
3. Do not enable anonymous, phone, social, SAML, or OIDC providers without product, billing, and privacy approval.
4. Review **Settings > Authorized domains** and remove obsolete entries.
5. Configure password policy, email templates, sender identity, and action URLs.
6. Test registration, email verification, reset, forced password change, disabled accounts, and role routing.

Authentication proves identity; it does not assign an LMS role. Every Auth account needs a coherent `users/{uid}` profile.

10.7 Create Cloud Firestore

1. Open **Build > Firestore Database** and select **Create database**.
2. Use **Native mode**.
3. Select `asia-southeast1` to match the checked-in Functions region and current architecture unless a formal migration design changes every coupled resource. Location is a foundational choice that is difficult to change.
4. Start with locked/production rules, never open test mode.
5. Deploy the repository rules and indexes:

```
firebase deploy --only firestore:rules,firestore:indexes --project staging
```

6. Run the authorization tests.
7. Repeat for production only through the approved release workflow.

Firestore creates collections on the first authorized write; do not manually create every path. Bootstrap `config/appSettings` through the admin UI where possible. If manual creation is unavoidable, use `DEFAULT_APP_SETTINGS` in `firestoreService.js` as the schema source.

10.8 Create Cloud Storage

1. Confirm Blaze billing.
2. Open **Build > Storage** and click **Get started**.
3. Choose a location consistent with the residency decision and close to the database, Functions, and users. Keep Southeast Asia for this architecture unless an approved design changes it.
4. Never leave the bucket in public/test mode.
5. Deploy the checked-in rules:

```
firebase deploy --only storage --project staging
```

6. Test allowed and denied avatar, class-file, submission, and branding cases.
7. Review soft delete/versioning, lifecycle, CORS, retention, and backup settings in Google Cloud Console.

Some US bucket locations have an Always Free allowance; other regions follow Google Cloud Storage pricing. Do not move learner files to a cheaper region without a residency, latency, and governance decision.

10.9 Configure App Check

The current client uses reCAPTCHA v3:

1. create a separate reCAPTCHA registration for each environment and its exact domains;
2. in **Build > App Check**, select the Web app and register the reCAPTCHA v3 provider using the server-side secret;
3. place only the public site key in `VITE_RECAPTCHA_SITE_KEY`;
4. deploy to staging with product enforcement initially off;
5. exercise every role and callable and monitor App Check metrics;
6. enable enforcement one product at a time for Functions, Firestore, and Storage, with rollback instructions and monitoring.

Firebase now recommends reCAPTCHA Enterprise for new web integrations and provides a no-cost assessment allowance before usage charges. Migrating this repository requires changing `firebase.js` to the Enterprise provider and retesting enforcement. It is not a console-only switch.

Never place a reCAPTCHA secret key or App Check debug token in a `VITE_*` variable, source file, guide, or client-visible CI variable.

10.10 Deploy Functions

1. Confirm Node.js 22 and Blaze billing.
2. Install and validate:

```
npm ci --prefix functions
node --check functions/src/index.js
node -e "require('./functions/src/index.js')"
```

3. Deploy:

```
firebase deploy --only functions --project staging
```

4. Confirm every expected export appears in `asia-southeast1`.
5. Test Auth, App Check, role rejection, assessment submission, progress, enrollment, graduation, verification, deletion, and trigger delivery.
6. Review logs, instance limits, duration, error rate, and artifact retention.

10.11 Create Hosting

```
npm run build:staging
firebase deploy --only hosting --project staging
```

Verify SPA deep links, security headers, caching, Auth domains, App Check, uploads, and callable region from the deployed URL. Add the approved production custom domain and DNS records through Hosting and wait for TLS provisioning before launch.

10.12 Bootstrap the first administrator

The first admin is a controlled break-glass bootstrap:

1. obtain written authorization naming the person and environment;
2. create the Auth account in Firebase Console using that person's organization-controlled email;
3. prefer a password-reset/setup flow instead of sharing a reusable password;
4. copy the generated UID;
5. create `users/{uid}` in Firestore with the minimal fields expected by `createUserProfile`, including the same UID, normalized email, active status, and `role: "admin"`;
6. sign in and verify the admin workflow;
7. create later admins through the supported application;
8. record approval, actor, UID, timestamp, and bootstrap fields in a restricted operations log.

Never create a hardcoded default admin or leave a shared bootstrap account.

10.13 Configure CI/CD without distributing keys

Recommended target:

1. create protected GitHub environments for staging QA and production;
2. require named production reviewers;
3. use GitHub OpenID Connect with Google Cloud Workload Identity Federation and a least-privileged deployment service account;
4. grant only required deployment roles;
5. store public web configuration as protected environment variables;
6. store staging test credentials as environment secrets and rotate them;
7. do not download a service-account JSON key for CI;
8. remove `FIREBASE_TOKEN` after workload-identity deployment is proven.

The current workflow still expects `FIREBASE_TOKEN`; migration remains a hardening task. Never email or document that token.

10.14 Seed and accept staging

```
npm run qa:seed:identities
npm run qa:seed:data
npm run qa:verify-seed
npm run test:qa:full
```

Then test the deployed URL, confirm indexes, Rules and App Check denials, verify billing-alert recipients, record resource regions, and perform a backup/restore exercise. Document the environment without copying a key, token, password, or service-account material into the repository.

10.15 Official setup and pricing references

- [Firebase pricing plans](#)
- [Firebase product pricing](#)
- [Firebase web API key management](#)
- [Cloud Firestore billing](#)
- [Cloud Storage web setup](#)
- [Cloud Functions quotas](#)
- [Firebase App Check for web](#)
- [Google Cloud budgets and alerts](#)

11. Development workflows

Common commands

Command	Purpose
<code>npm run dev</code>	Default Vite environment; verify project first
<code>npm run dev:staging</code>	Staging-configured Vite server
<code>npm run build</code>	Production/default bundle
<code>npm run build:staging</code>	Staging bundle
<code>npm run preview</code>	Serve built <code>dist</code>
<code>npm run test:unit</code>	Vitest unit tests
<code>npm run test:rules</code>	Firestore and Storage authorization tests
<code>npm run test:e2e</code>	Chromium E2E
<code>npm run test:qa:full</code>	Validated all-project Playwright run
<code>npm run test:e2e:responsive</code>	Responsive checks
<code>npm run test:e2e:roles</code>	Authenticated role routes
<code>npm run test:e2e:ui</code>	Interactive Playwright runner
<code>npm run test:e2e:report</code>	Open last HTML report
<code>npm run qa:seed:identities</code>	Create/update staging test identities
<code>npm run qa:seed:data</code>	Seed disposable staging fixtures
<code>npm run qa:verify-seed</code>	Validate seeded fixtures
<code>npm run qa:cleanup</code>	Dry-run QA cleanup
<code>npm run qa:cleanup:apply</code>	Apply QA cleanup
<code>npm run backup:dump</code>	Point-in-time project data dump
<code>npm run audit:academic</code>	Dry-run trusted-data audit
<code>npm run migrate:academic</code>	Apply approved academic migration

There is no configured ESLint script. Do not tell a new maintainer that `npm run lint` is part of the current gate.

Change checklist by domain

For a UI-only change:

1. exercise all affected roles and empty/loading/error states;
2. test 360 px mobile through desktop widths;
3. run public/role/responsive E2E as applicable;
4. build staging.

For Firestore/Storage changes:

1. change the service call and rule together;
2. add allow and deny rules tests;
3. confirm required indexes;
4. test an adjacent role cannot access the data;
5. deploy rules before code that depends on them only under a planned compatible release sequence.

For Functions/academic changes:

1. add caller, role, status, ownership, and input tests;
2. keep private data server-owned;
3. test concurrency/attempt limits and both assessment kinds;
4. run syntax and module-load checks;
5. validate first on staging with App Check;
6. inspect Function logs and security records after deployment.

Index changes

Firestore query errors often include a generated index link. Add required indexes to `firestore.indexes.json`; never create an undocumented production-only index and leave source control behind.

12. QA and test strategy

Automated coverage

Current automation includes:

- CSV formula-injection hardening unit tests;
- Firestore and Storage authorization tests;
- repository secret scanning;
- Functions syntax and runtime-load validation;
- production/staging builds;
- public landing/sign-in/sign-up smoke checks;
- password visibility behavior;
- protected-route redirect checks;
- role dashboard and cross-role redirect checks;
- axe serious/critical accessibility checks;
- responsive overflow/control checks;
- Chrome, Edge, Firefox, and WebKit projects.

Automated coverage is not complete product coverage. Destructive operations, assessment variants, file workflows, academic decisions, certificate operations, and complex role interactions still require the manual playbook and targeted tests.

Evidence locations

- Playwright HTML: `HYTech/playwright-report/`
- traces/screenshots/videos: `HYTech/test-results/`
- CI failure artifact retention: 14 days

These directories are ignored by Git.

CI behavior

`.github/workflows/qa.yml` has two jobs:

1. security/unit/rules/Functions-load/build/dependency checks;
2. seeded staging-backed Playwright testing across browser projects.

The Playwright CI job serves a staging-configured build locally at `127.0.0.1:4173`. It talks to staging Firebase, but it does **not** prove the currently deployed staging Hosting site. Before production, separately run against:

```
E2E_BASE_URL=https://hytech-lms-staging.web.app
```

Dependency audits are currently `continue-on-error`; review their output even when CI is green.

13. Deployment and release

Staging deployment

Hosting-only helper:

```
npm run deploy:staging
```

This does not deploy rules, indexes, Storage rules, or Functions.

For a complete staging release:

```
npm run build:staging
```

```
firebase deploy --project staging
```

Or deploy explicit resources:

```
firebase deploy --only firestore:rules,firestore:indexes --project staging
```

```
firebase deploy --only storage --project staging
```

```
firebase deploy --only functions --project staging
```

```
firebase deploy --only hosting --project staging
```

Functions deployment may require Blaze billing. Obtain owner approval before attaching billing. Confirm the CLI project in command output before approval.

Production CI/CD

`.github/workflows/deploy.yml`:

- runs after successful QA on `main`;
- permits manual `workflow_dispatch` on `main` as a rollback/redeploy path;
- uses the protected GitHub `production` environment;
- installs both lockfiles under Node 22;
- builds with production Firebase/App Check configuration;
- fails if the App Check site key is absent;
- performs a Firebase dry run;
- configures one-day Functions artifact cleanup;
- deploys all resources declared in `firebase.json`.

“All resources” means Hosting, Firestore rules/indexes, Storage rules, and Functions. It is not a Hosting-only workflow.

The manual dispatch path bypasses the QA workflow dependency, although production environment approval still applies. Use it only for an understood rollback or known-good redeployment.

Pre-production release gate

1. Confirm the exact commit and a clean build.
2. Review `TURNOVER_RISK_REGISTER.md`.
3. Run secret, unit, rule, module-load, build, and E2E gates.
4. Deploy all changed resources to staging.
5. Test the deployed staging URL, not just local preview.
6. Exercise admin, trainer, active/pending/unenrolled/disabled student cases.
7. Verify enrollment isolation, assessment grading, manual grading, progress, graduation/certificate, uploads, CSV, CSP, and App Check.
8. Review Function errors and Security Rules denials.
9. Approve the protected production environment.
10. Record deployed commit, operator, timestamp, and post-deploy evidence.

Rollback

- Hosting: redeploy a known-good commit/version.
 - Rules/indexes/Functions: check out a known-good commit and deploy the explicit resource set.
 - Data: do not “roll back” with ad hoc client writes. Use PITR/export/approved repair and preserve an audit trail.
 - Academic records: correction/revocation requires academic authorization, not only engineering approval.
-

14. Operational runbooks

User onboarding

- Self-registration creates a student profile and requires the intended email verification path.
- Trainer/admin creation is an administrative operation.
- Admin-created users may be forced to change their password.
- A Firebase Auth user without a coherent `users/{uid}` document will not have a valid application authorization state.

User offboarding

1. Disable access before deleting anything.
2. Revoke Firebase refresh tokens.
3. transfer class ownership and operational responsibility;
4. identify enrollments, profiles, private data, preferences, progress, submissions, activity, certificates, and files subject to retention;
5. obtain privacy/academic approval for deletion;
6. perform controlled cleanup and retain evidence.

There is no active Auth `onDelete` cleanup export. Deleting only the Auth record can orphan Firestore data.

Assessment incident

If answers, attempts, or scores appear compromised:

1. unpublish/close the affected item if safe;
2. preserve the public item, private answer-key, attempt, enrollment, security, and activity records;
3. identify whether it is under `assessments` or `assignments` ;
4. review Function logs and attempt timing/limits;
5. do not edit attempts directly from the browser;
6. use an approved regrade/reissue workflow and notify academic ownership.

Upload incident

1. restrict access or remove the referencing document;
2. preserve metadata and logs if investigation is required;
3. inspect the Storage path and owner/class scope;
4. treat file content as untrusted because no malware scan exists;
5. remove/quarantine through an approved operator workflow.

Backup and restore

The repository contains `scripts/dump-project-data.mjs` , while live Firestore backup/PITR state is external. Follow the latest risk register.

- define approved RPO and RTO;
- keep exports outside the repository in restricted storage;
- include Auth dependencies and Storage, not only Firestore documents;
- restore into a nonproduction project;
- verify rules, indexes, files, profiles, enrollment, attempts, progress, certificates, and logs;
- exercise restoration at least quarterly;
- record duration and gaps.

An untested backup is not a verified recovery capability.

Academic integrity audit/migration

Both scripts require privileged credentials through environment variables:

```
$env:FIREBASE_SERVICE_ACCOUNT_JSON = '<authorized-json>'
$env:GOOGLE_CLOUD_PROJECT = 'hytech-lms-staging'
npm run --silent audit:academic
```

Required sequence:

1. back up Firestore and Storage;
2. run the dry-run audit;
3. confirm `projectId` and `mode` in output;
4. review inline answer keys, unverified progress, and legacy certificates;
5. obtain approval;
6. run `npm run migrate:academic` ;
7. recalculate through trusted Functions;
8. repeat the dry run until expected findings are clear.

Use `--silent` when redirecting JSON so npm banners do not corrupt the file. Never put service-account JSON or output in the repository.

App Check rollout

Callable wrappers enforce App Check. Firestore/Storage enforcement is configured in the Firebase console and may differ by environment.

1. register separate staging and production reCAPTCHA/App Check apps;
2. configure matching site keys;
3. observe staging metrics before enforcement;
4. enforce and test one Firebase product at a time;
5. monitor legitimate and rejected traffic;
6. never ship a production debug token.

Check the dated risk register for the live enforcement status.

15. Troubleshooting

Wrong Firebase project

Symptoms include unexpected data, missing users, permission errors, or a deploy targeting production.

```
firebase use
firebase projects:list
```

Check Vite mode and the `VITE_FIREBASE_PROJECT_ID` compiled into the build. Remember that `.firebaserc` defaults to production.

Missing or insufficient permissions

Check, in order:

1. Firebase Auth session;
2. `users/{uid}` existence;
3. normalized active status and exact role;
4. class ownership/co-trainer/membership;
5. query constraints versus rules;
6. deployed rule version and project;
7. App Check enforcement/attestation;
8. browser console and Firebase logs.

Do not weaken a rule until the denied request and intended authorization are understood and covered by a deny test.

Callable fails with unauthenticated/permission denied

- confirm the browser and Function use the same Firebase project;
- verify Auth token refresh and App Check;
- check the caller profile and active state;
- check the Function region (`asia-southeast1`);

- inspect Function logs and `securityLogs` where applicable.

Quiz submission or grading fails

- identify `assessment` versus `assignment`;
- verify published/available/due/attempt/time-limit state;
- verify enrollment is active/ongoing;
- confirm `private/answerKey`;
- check `submitAssessmentAttempt` logs;
- do not fall back to client-side authoritative scoring.

Rules tests fail to start

- install Java 21;
- confirm ports 8080 and 9199 are free;
- run from `HYTech/`;
- use the pinned script rather than a globally mismatched CLI.

Functions syntax passes but deployment/load fails

Run both:

```
node --check functions/src/index.js
node -e "require('./functions/src/index.js')"
```

The second command catches runtime API incompatibility.

Playwright role tests skip or fail validation

- fill `.env.e2e.local`;
- use staging-only accounts;
- confirm profile role/status and required email verification;
- seed/verify fixtures;
- set `E2E_REQUIRE_AUTH=true` when a complete role gate is required.

CSP blocks a new resource

Find the exact directive and origin in the console. Add only the required origin to `firebase.json`, deploy Hosting to staging, and verify headers there.

16. Known risks and engineering priorities

The authoritative dated list is `HYTech/docs/TURNOVER_RISK_REGISTER.md`. At this turnover, maintainers must explicitly review:

- App Check enforcement state for production Firestore and Storage;
- live test or malformed accounts in production;
- historical credentials present in public Git history;
- Cloud Data Access audit logging coverage;
- unverified legacy academic progress;
- orphaned data after Auth deletion;

- absence of upload malware scanning;
- unexercised restore procedure and undefined/unsigned RPO/RTO;
- long-lived Firebase deployment credential instead of WIF/OIDC;
- operator workstation service-account keys;
- Storage soft-delete configuration;
- outstanding privacy, academic, accessibility, incident, and retention governance sign-offs.

Engineering priorities:

1. close high-severity operational risks with retained evidence;
 2. implement a governed user offboarding/cleanup path;
 3. exercise a full nonproduction restore;
 4. expand trusted academic workflow tests;
 5. replace long-lived CI deployment credentials with workload identity;
 6. modularize `ClassDetail`, `StudentCourse`, `firestoreService`, and Functions;
 7. decide whether to expose, complete, or remove the dormant certificates UI;
 8. add content scanning before allowing untrusted public uploads;
 9. add linting/static analysis as an explicit, stable CI gate;
 10. keep dependency risk dispositions owned and time-bound.
-

17. Handover acceptance checklist

Repository and engineering

- ☐ Repository access transferred with branch protection understood.
- ☐ Receiving developer can install with both lockfiles.
- ☐ Staging build, unit tests, rules tests, and Functions load pass.
- ☐ Receiving developer has read the four maintainability hotspots.
- ☐ No local secret or production export is present in the repository.

Firebase and environments

- ☐ Production and staging project ownership is assigned.
- ☐ Auth, Firestore, Storage, Functions, Hosting, and App Check access is least-privileged.
- ☐ Region and billing responsibility are understood.
- ☐ App Check enforcement state is recorded from the live console.
- ☐ Firestore backup, PITR, delete protection, and Storage protection state are verified live.

CI/CD and credentials

- ☐ GitHub `staging-qa` and `production` environments have named owners.
- ☐ Production reviewers and manual-dispatch responsibility are agreed.
- ☐ Staging QA accounts have owners and rotation/offboarding dates.
- ☐ `FIREBASE_TOKEN` migration to WIF has an owner.
- ☐ Operator service-account keys are inventoried and time-limited.

Operations and governance

- ☐ Open risks have named owners and deadlines.

- ☐ Incident response and escalation contacts are recorded outside this repo.
- ☐ RPO/RTO are approved and a restore test is scheduled.
- ☐ Retention/deletion and academic correction/revocation are approved.
- ☐ Privacy, accessibility, TESDA/TVET, and processor obligations have owners.
- ☐ Production test accounts and historical credential exposure are resolved or formally accepted.

Acceptance

Handed over by: _____

Role: _____

Date: _____

Accepted by: _____

Role: _____

Date: _____

Open-risk register reviewed: Yes / No

Production access verified: Yes / No

Restore evidence reviewed: Yes / No

18. Document maintenance

Update this guide when any of the following changes:

- role model or route table;
- Firebase project, region, or product configuration;
- data paths or authorization rules;
- Function exports, runtime, scaling, or App Check behavior;
- academic progress, grading, graduation, or certificate authority;
- Storage paths, file limits, or accepted content types;
- test commands or CI/CD gates;
- backup, incident, credential, or deployment procedures.

For every update:

1. audit source and live operational state separately;
2. update the Markdown source;
3. regenerate the PDF;
4. verify commands, links, headings, page breaks, and document date;
5. commit both formats in the same change.

The source code, rules, deployment workflow, and live Firebase configuration remain authoritative over this narrative.